

CLAUDE CERTIFIED DEVELOPER — FOUNDATIONS

Key Terms & Acronyms Cram Sheet

KEY TERMS

Workflow vs. Agent A workflow is a developer-authored sequence of steps where control flow is fixed at design time

Manager / Subagent Hierarchy (Orchestrator-Worker) A multi-agent pattern where a manager (orchestrator) agent decomposes a goal, delegates scoped subtasks to subagents...

Context Isolation via Subagents Delegating a subtask to a subagent so its intermediate reasoning and tool output stay in a separate context window...

query() (Claude Agent SDK) The single entry point of the Claude Agent SDK (Python `claude-agent-sdk`, TS `@anthropic-ai/claude-agent-sdk`) that runs...

Agent Loop The repeating cycle the Agent SDK runs: send context to the model, receive a `tool_use` or final answer, execute tools...

Agent SDK Input Modes (streaming vs. single-message) The Agent SDK accepts input two ways: streaming input (the default and recommended mode) and single-message input

Custom Agent Loop / Harness Hand-writing the reason-act-observe loop directly against the Messages API instead of using the Agent SDK: the...

AgentDefinition The Agent SDK config object describing an agent

Managed Agent Deployment: Self-Hosted vs. Anthropic-Hosted Two managed deployment shapes

Hooks (Lifecycle Events) Deterministic scripts registered in `settings.json` that fire at agent lifecycle events: `PreToolUse`, `PostToolUse`...

PreToolUse permissionDecision The field a `PreToolUse` hook sets (inside `hookSpecificOutput`) to control a pending tool call

Hooks as Guardrails Using a `PreToolUse` hook to deny or ask on a dangerous pending action (for example a destructive shell command) before...

LangGraph A LangChain framework built as an explicit graph / state machine

StateGraph LangGraph's core primitive: a graph parameterized by a user-defined State object, with nodes and edges added via...

Reducers (LangGraph) In LangGraph, a reducer is a binary function (`current`, `update`) returning the new value, defining how a node's partial...

Checkpointers (LangGraph) LangGraph's thread-scoped persistence layer providing continuity, human-in-the-loop, time-travel, and fault tolerance

Strands An AWS Apache-2.0 framework (Python, TS preview) built as a model-driven agent loop from three components: model...

PydanticAI A framework from the Pydantic team built as a model-driven agent loop differentiated by Pydantic-validated structured...

Framework Cross-Attribution (LangGraph / Strands / PydanticAI) The three are not interchangeable: LangGraph is the explicit graph/state-machine one

Agent Design Patterns (memory, context-window management) Common agent-building patterns: tool-use loops, subagents for context isolation, external memory for durable state, and...

Functional vs. Infrastructure Requirements Functional requirements state what the application must do (behaviors derived from business needs)

Systems Life Cycle (Develop, Implement, Operate, Maintain) The phased life cycle of a Claude application: develop (build and integrate), implement (deploy), operate (run in...

Messages API Structure The core Claude API request: a model, a system prompt, a messages array of role-tagged turns (user/assistant) whose...

Client Tools vs. Server Tools Client tools (developer-defined plus Anthropic-schema `bash`, `text_editor`, `memory`, `computer_use`) run in the developer's...

tool_choice Values `tool_choice` controls whether Claude calls a tool

stop_reason: tool_use The response `stop_reason` signaling that Claude emitted a client tool call and is pausing for the developer to execute...

Streaming SSE Event Order With `stream: true`, events follow a fixed order: `message_start`, then per block content `block_start`, one or more...

tool_use Streaming (partial JSON) During streaming, a `tool_use` block's arguments arrive as partial JSON: deltas of type `input_json_delta` carrying a...

Vision (Image Input) Claude accepts images as content blocks in the messages array alongside text, enabling multimodal prompts

Adaptive Thinking The current extended-thinking path: thinking.type: adaptive with no budget_tokens

effort (output_config.effort) The parameter controlling reasoning effort, nested at output_config.effort

Legacy budget_tokens (thinking.type: enabled) The legacy extended-thinking mode: thinking.type: enabled with budget_tokens (which must be less than max_tokens...

Thinking Blocks in Multi-Turn During multi-turn or tool use, thinking and redacted_thinking blocks (including the encrypted signature) in the latest...

Prompt Caching Breakpoints Up to four explicit cache_control breakpoints per request

Cache TTLs and Cost Multipliers Prompt caching offers a 5-minute ephemeral TTL (default) and a 1-hour TTL ({"type":"ephemeral","ttl":"1h"})

Message Batches API Asynchronous processing at 50% of standard price

custom_id (Batches) A required unique identifier on each batch request matching `^[a-zA-Z0-9_-]{1,64}$`

Batches Supported vs. Unsupported Params Batches DO support tool use (including server tools), multi-turn, vision, system messages, and extended thinking

Bedrock Current Integration (bedrock-mantle) The current Amazon Bedrock path: the Messages API at `https://bedrock-mantle.{region}.api.aws/anthropic/v1/messages...`

Bedrock Legacy Integration (InvokeModel / us.anthropic.*) The legacy Bedrock path using InvokeModel or Converse with region-scoped inference-profile IDs (us.anthropic.*) and the...

AnthropicBedrockMantle vs. AnthropicBedrock AnthropicBedrockMantle is the SDK client for the current bedrock-mantle Messages endpoint

Google Vertex AI Integration Invoking Claude via Vertex: the model ID goes in the endpoint URL (not the body), and authentication uses Google Cloud...

anthropic_version on Vertex On Vertex the anthropic_version field goes in the request BODY (not a header) and must equal the literal...

REST API and JSON The Claude API is an HTTP REST interface exchanging JSON: requests carry a JSON body (model, messages, params) and...

Asynchronous Programming (API integration) Non-blocking patterns (async/await, concurrent requests, streaming consumers) that let an app issue and await Claude...

Version Control and SDLC Integration (git) Using git and standard SDLC practices to manage a Claude application's source, including checking configuration such as...

Instruction Interpretation Across Interfaces The same underlying model is reached through different surfaces (Claude Code, Claude Desktop, claude.ai, the API, and...

Content Boundaries Explicit delineation of trusted developer instructions versus untrusted external or user content inside a prompt (for...

Schema Design (tools and structured output) Designing tool input schemas and structured-output schemas with precise field names, types, required vs

Session Hygiene Managing conversation state deliberately: pruning stale tool output, compacting or summarizing long histories, and...

Plugin Management Installing, configuring, and managing plugins that extend an application or Claude Code with additional commands...

CLAUDE.md Hierarchy Markdown instruction files loaded automatically into context, layered by scope: user-level (~/.claude/CLAUDE.md)...

settings.json Configuration The JSON configuration file (~/.claude/settings.json, .claude/settings.json, settings.local.json) that holds...

Model Version Pinning Specifying an exact model ID (for example claude-opus-4-8) rather than a floating alias so application behavior stays...

Prompt Versioning Treating prompts as versioned artifacts (tracked in source control, tied to a model version, and regression-tested) so...

Rules (Claude Code) Standing, always-on instructions that shape Claude Code's behavior in a project, expressed in CLAUDE.md memory files...

Skills (Claude Code) Reusable capabilities packaged as a SKILL.md file with YAML frontmatter (name, description) plus Markdown instructions...

Commands (Slash Commands) Slash-prefixed shortcuts typed in the Claude Code session

Agents / Subagents (Claude Code) Separate Claude Code agent instances spawned to handle a delegated subtask in an isolated context with their own tools...

Agent Memory Persistent state an agent maintains across turns or sessions outside the live context window, for example the...

Repo Init (/init) The built-in /init slash command that scans the current repository and generates a starter CLAUDE.md capturing...

settings.json The JSON configuration file controlling Claude Code permissions, hooks, environment variables, and model selection

Headless Mode (-p / --print) Non-interactive execution invoked with the -p or --print flag: Claude Code takes a prompt, runs the task to completion...

Streaming Mode (Claude Code) Input/output modes for programmatic Claude Code use

Auto Mode / Accept-Edits An operating mode where Claude Code executes and applies actions (file edits, allowed tool calls) without pausing for...

Session Management (Claude Code) Controlling conversation state across a session: compaction when context fills, /clear to reset, and resume/continue to...

Rules vs Skills vs Commands vs Agents The four Claude Code extension mechanisms: Rules are always-on CLAUDE.md instructions

Error Type Identification The diagnostic step of classifying what kind of failure occurred: a transient infrastructure/tool error, a permanent...

Recovery Strategy Selection Choosing the response given an identified error type: retry with backoff for transient errors, fallback or escalation...

Transient Error vs Permanent Error Transient errors (timeouts, rate limits, 5xx, network blips) are expected to clear on their own and are retrievable...

Reasoning Error vs Tool/Integration Error A reasoning error originates in the model output: a wrong plan, misread result, or bad tool argument despite sufficient...

Problem-Origin Isolation The debugging discipline of localizing a failure to a specific layer, the integration/harness code versus the model...

Trace Analysis Reading the ordered execution record, prompts, tool calls, tool results, thinking/response blocks, and errors, to...

Silent Failure (Debugging) A response that returns success with no error signal but is wrong, incomplete, or malformed

Refusal vs Bug A refusal is the model intentionally declining a request on policy or safety grounds, a valid model behavior

Token The atomic unit of text the model reads and generates, a sub-word chunk roughly a few characters long

Context Window The maximum number of tokens a model can attend to in a single request, spanning system prompt, conversation history...

Sampling The process of selecting each next token from the model's output probability distribution

Non-Determinism The property that the same prompt can yield different outputs across calls, because next-token selection is...

Next-Token Generation The autoregressive mechanism by which a language model produces text one token at a time, each token conditioned on all...

Fast Mode (speed) A latency-optimized execution option (the speed parameter) that prioritizes response time

Extended Thinking (legacy config) The older thinking configuration set via thinking.type:"enabled" plus an explicit budget_tokens reasoning budget

Effort Levels (output_config.effort) The control for how much work the model spends on a response, nested at output_config.effort with values max, xhigh...

Zero-Shot Prompting Giving the model a task instruction with no worked examples, relying on pretrained knowledge and instruction clarity

Single-Shot (One-Shot) Prompting Providing exactly one input/output example before the task

Multi-Shot (Few-Shot) Prompting Supplying two or more input/output examples to demonstrate format, tone, or reasoning

SDKs Wrap the REST API The Anthropic SDKs (Python anthropic, TypeScript @anthropic-ai/sdk, plus Bedrock/Vertex variants) are convenience...

Websockets vs SSE Streaming Claude streaming is delivered as Server-Sent Events (SSE) over a single HTTP response when stream:true is set, emitting...

Opus (model tier) Anthropic's most capable model tier (for example Opus 4.8), best for the hardest reasoning, coding, and agentic tasks

Sonnet (model tier) The balanced model tier (for example Sonnet 5) trading capability against speed and cost

Haiku (model tier) The fastest, lowest-cost model tier, suited to high-throughput, latency-sensitive, or simple tasks such as...

Opus vs Sonnet vs Haiku (tradeoff) The three tiers trade quality against latency and cost: Opus highest quality/cost, Sonnet balanced, Haiku...

Breaking Behavior Changes Across Releases Newer model generations can reject parameters that older ones accepted

Token Usage Tracking and Cost Modeling Reading the usage fields returned on each response (input_tokens, output_tokens, cache_creation_input_tokens...

Prompt Caching Reusing a previously processed prompt prefix by marking it with cache_control, so repeated large, stable contexts...

Cache TTL (5-minute vs 1-hour) Prompt cache entries have two lifetimes: the default 5-minute ephemeral TTL and an optional 1-hour TTL set with...

Cache Cost Multipliers Relative to base input price: a cache READ costs 0.1x, a 5-minute cache WRITE costs 1.25x, and a 1-hour cache WRITE...

Cache Check-Pointing (Breakpoints) Placing up to 4 explicit cache_control breakpoints in a request to mark cacheable prefix boundaries

Realtime vs Batch (cost tradeoff) The synchronous Messages API returns immediately at standard price

Context Window Management The discipline of deliberately controlling what occupies a session's finite token budget so the model keeps the...

Context Drift The gradual loss of task focus in a long session as accumulated turns, tangents, and stale results dilute the...

Context Bloat The condition where a context window fills with low-value tokens, verbose tool dumps, redundant history, or filler...

Tool-Output Pruning The practice of trimming, summarizing, or dropping verbose tool results before they accumulate in context, keeping only...

Context Compaction Periodically compressing earlier turns into a shorter summary that preserves the essential thread while reclaiming...

Context Isolation (Subagents) Delegating a scoped subtask to a subagent that receives only the context it needs and returns only its result, keeping...

Instruction Clarity Writing prompt instructions that are specific, unambiguous, and ordered, stating exactly what to do, what format to...

System vs User Prompt Placement The decision of where content belongs: the system prompt sets persistent role, behavior, and constraints for the whole...

Output Constraints Explicit limits placed in the prompt on the shape of the response: required format, allowed values, length bounds, or...

Few-Shot Examples Including a small number of high-quality input-output examples in the prompt to demonstrate the desired format, tone...

Iterative Refinement Improving a prompt through cycles of test, observe failures, and adjust, ideally measured against a fixed evaluation...

Input Sanitization Cleaning, validating, or clearly delimiting user-supplied content before it enters the prompt so that data is treated...

Structured Output Patterns Techniques for making the model emit machine-readable output: structured outputs or strict tool use for guaranteed...

Response Validation Programmatically checking a model response against a schema or business rules after generation, and on failure...

Defensive Parsing Consuming model output with code that anticipates malformed, incomplete, or unexpected content: tolerant extraction...

Skepticism Toward Confident Output The stance that a fluent, self-assured model response is not evidence of correctness, because models state wrong...

Prompt Injection An attack in which untrusted content reaching the model, from a web page, document, tool result, or user field, carries...

Jailbreak An attempt to coax the model past its own safety guidelines and content policy so it produces disallowed output, using...

Prompt Injection vs Jailbreak Two distinct threats: prompt injection hijacks the application by planting instructions in untrusted input the model...

Untrusted Input Any content the application did not author and cannot vouch for: end-user messages, retrieved documents, web content...

Data Leakage The unintended exposure of sensitive information, secrets, other users' data, system-prompt contents, or internal...

PII Handling Controls for personally identifiable information across its lifecycle: minimize what is collected, redact or tokenize...

Confidentiality, Integrity, Privacy Three security properties an AI application must uphold: confidentiality keeps data from unauthorized parties...

Authentication vs Authorization Authentication (AuthN) verifies WHO a caller is

Guardrail Layering Placing safety controls at multiple points, input guardrails that screen or delimit incoming content, output guardrails...

Content Policy The defined rules for what an application will and will not produce or act on, enforced through model safety behavior...

Secure-by-Design Building security in from the start, applying privacy, identity, and access controls as architectural defaults rather...

Least Privilege Granting an agent, tool, or identity only the minimum capabilities and access needed for its function, and REMOVING...

IAM (Identity and Access Management) The systems and policies that establish identities and control what each identity may access, scoped per user, per...

Secrets and API Key Management Keeping credentials, API keys, and tokens out of source code and prompts, loading them from environment variables or a...

Identity Validation and Access Monitoring Confirming a caller's identity before granting a privileged action, requiring approval where warranted, and...

Tool Use (Function Calling) The capability by which the model, given tool definitions, emits a structured request to call a named tool with typed...

Tool Description The natural-language text in a tool definition explaining what the tool does, when to use it, and what it returns

Tool Schema The JSON Schema attached to a tool that declares its input parameters, types, required fields, and enum constraints

Tool Result The message the developer returns to the model after executing a client-side tool call, referencing the call by its...

Base Tool-Use Token Cost The system-prompt overhead (roughly 290 tokens on Opus 4.8) that is added whenever tools are provided in a request...

Tool Error Handling Returning tool failures as structured, actionable responses, an error type and readable message, and surfacing partial...

Client-Side Tools Tools that execute in the developer's own application: user-defined functions plus Anthropic-schema tools such as bash...

Server-Side Tools Tools that run on Anthropic's infrastructure, such as web_search, web_fetch, code_execution, and tool_search, where no...

Client-Side vs Server-Side Tools The execution-location distinction: client-side tools run in the developer's app and require the developer to return a...

tool_choice The request parameter controlling whether and how the model calls tools, with exactly four values: auto (default, model...

Approval Patterns (Tool Use) Requiring human confirmation before an agent executes a sensitive or irreversible tool action, implemented via a...

Tool Set Construction Designing the collection of tools an agent is given: few, well-scoped, clearly described, non-overlapping tools rather...

MCP (Model Context Protocol) An open, model-agnostic standard defining how applications connect to external capabilities through a uniform...

MCP Primitives (Tools, Resources, Prompts) The three capabilities an MCP server exposes, distinguished by who controls them: tools are model-controlled (the model...

MCP Server Authoring Building an MCP server that registers tools, resources, and prompts and serves them to clients over a transport per the...

stdio Transport (MCP) An MCP transport where the client spawns the server as a local child process and they exchange messages over standard...

Socket / HTTP Transport (MCP) An MCP transport where the server runs as a networked service reachable over sockets or HTTP, supporting...

stdio vs Sockets (MCP Transports) The MCP transport selection rule: stdio spawns the server as a local child process for single-machine single-user use...

MCP Roots (Filesystem Access) The MCP mechanism by which a client explicitly grants a server access to specific filesystem paths or URIs

MCP Authentication How an MCP client proves identity to a remote MCP server before using its tools and resources

Built-in Tools vs Custom Tools vs Skills vs MCPs Four ways to extend an agent, chosen by tradeoff: built-in tools are ready-made Anthropic capabilities, custom tools...

ACRONYMS

AI Artificial Intelligence
API Application Programming Interface
CI/CD Continuous Integration and Continuous Delivery
CLI Command Line Interface
HTTP Hypertext Transfer Protocol
HTTPS Hypertext Transfer Protocol Secure
IAM Identity and Access Management
JSON JavaScript Object Notation
LLM Large Language Model
MCP Model Context Protocol
PII Personally Identifiable Information
REST Representational State Transfer
SDK Software Development Kit
SDLC Software Development Life Cycle
SLA Service Level Agreement
SSE Server-Sent Events
TTL Time To Live
URL Uniform Resource Locator
YAML YAML Ain't Markup Language